

# Amazon + Google — Microservices Interview Q&A (Java/Spring, AWS)

## How to use

- Treat this as **company-specific drill material**.
  - For every question:
    - State assumptions (scale, SLOs, constraints)
    - Cover failure modes + mitigation
    - Mention operational plan (dashboards, alarms, rollout)
- 

## Table of contents

1. Amazon (System Design)
  2. Amazon (Behavioral)
  3. Amazon (Deep Dive Debugging)
  4. Google (System Design)
  5. Google (Behavioral)
  6. Google (Deep Dive Debugging)
- 

## 1) Amazon — System Design (what to expect + questions + answer bullets)

Amazon interviews strongly emphasize: - **Operational excellence** (runbooks, oncall, alarms) - **Resilience** (timeouts, retries, bulkheads, idempotency) - **Cost awareness** (data transfer, managed services, right sizing) - **Ownership** (you drive ambiguity to decisions)

### 1.1 Common system design prompts

#### E-Commerce Orders / Checkout

- **Questions**
  - How do you model order state transitions?
  - How do you prevent duplicate orders/charges?
  - How do you implement refunds/chargebacks?
- **Answer bullets (what to say)**
  - Use a clear state machine; persist transitions; emit domain events.
  - Use **idempotency keys** + unique constraints.

- Prefer **saga** with compensation; avoid distributed transactions.
- Use outbox/inbox for event reliability.
- SLOs + alarms; canary rollout; graceful degradation.

### *High-signal detailed Q&A (Amazon)*

#### Q1: How do you prevent duplicate orders and duplicate charges?

- **Detailed answer**

- Make the *entire* write path idempotent:
  - Client sends Idempotency-Key.
  - Order service stores the key with a unique constraint (or a dedicated idempotency table).
  - If the same key is retried, return the original order result.
- Payment is also idempotent:
  - Pass the same idempotency key to payment provider when supported.
  - Store payment\_attempt\_id and enforce uniqueness per (order\_id, attempt).

- **Example**

- User taps “Pay” twice on bad network. The API receives two POST /orders calls with the same Idempotency-Key. The first creates the order and payment attempt; the second returns the already-created order without charging again.

- **Common follow-ups**

- Where do you store idempotency keys? TTL?
- What if the same key comes with a different payload?
- How do you handle client not sending the key?

#### Q2: Orchestration vs choreography for a saga?

- **Detailed answer**

- Prefer **orchestration** when:
  - You need a single place to observe/drive workflow state.
  - You want simpler operations and debugging.
- Prefer **choreography** when:
  - Workflows are simple and teams are truly independent.
  - You accept higher emergent complexity.

- **Example**

- Use AWS Step Functions (or an internal orchestrator) as the saga orchestrator: ReserveInventory → AuthorizePayment → CreateShipment, with compensations.

- **Common follow-ups**

- How do you implement compensation?
- What if a step succeeds but acknowledgment is lost?

Q3: How do you guarantee “DB write + event publish” correctness?

- **Detailed answer**
  - Use **transactional outbox**:
    - In the same DB transaction as the order write, insert an outbox row.
    - A relay publishes outbox rows to the broker and marks them published.
  - Consumers should be idempotent; expect at-least-once.
- **Example**
  - Order created in Aurora; outbox table row inserted. Relay publishes to Kafka/MSK. If relay crashes after publish but before marking, consumer idempotency prevents double processing.
- **Common follow-ups**
  - How do you handle outbox table growth?
  - How do you ensure ordering of events?

## Notification Platform (Email/SMS/Push)

- **Questions**
  - How do you handle retries, DLQs, and vendor rate limits?
  - How do you prevent duplicates?
- **Answer bullets**
  - Queue-per-channel/priority; DLQ + replay tooling.
  - Dedupe by notification-id/user-id; idempotent provider calls if possible.
  - Observability: delivery funnel metrics.

## High-signal detailed Q&A (Amazon)

Q4: How do you design retries without causing duplicate notifications?

- **Detailed answer**
  - Treat sending as at-least-once; prevent duplicates via idempotency:
    - Store notification\_id (or (user\_id, template\_id, dedupe\_window)) with unique constraint.
    - Ensure the “send attempt” is recorded before/after provider call depending on desired semantics.
  - Separate **retry** from main consumption:
    - Use retry queues/topics with backoff.
    - Use DLQ for poison payloads.
- **Example**
  - SQS consumer fails to call SMS provider due to timeout. It retries via a retry queue; dedupe table ensures the same notification\_id isn’t sent twice.
- **Common follow-ups**
  - How do you define dedupe window?

- How do you replay DLQ safely?

## Billing + Metering (multi-tenant)

- **Questions**

- How do you ensure no usage events are lost?
- How do you backfill/replay safely?

- **Answer bullets**

- Immutable raw event log (e.g., S3) + replay.
- Deterministic aggregation + versioned pipelines.
- Reconciliation reports; audit trails.

### *High-signal detailed Q&A (Amazon)*

#### Q5: How do you ensure no usage events are lost?

- **Detailed answer**

- Make ingestion durable before processing:
  - Append raw events to an immutable log (S3) or a durable stream (Kafka/Kinesis) with retention.
- Use at-least-once delivery and idempotent aggregation:
  - Use an event ID + dedupe in the aggregation store.
- Operate with explicit RPO/RTO for finance-grade correctness.

- **Example**

- Every usage event is written to S3 as the system of record. Aggregators read from stream, but if a bug happens you can backfill from S3.

- **Common follow-ups**

- How do you handle late-arriving events?
- How do you reconcile vendor totals?

## Rate Limiting + Abuse Detection

- **Questions**

- Fail-open vs fail-closed?
- How do you enforce tenant quotas?

- **Answer bullets**

- Layered limits (edge coarse + app fine).
- Decide fail-open/closed by risk: auth vs optional features.

### *High-signal detailed Q&A (Amazon)*

#### Q6: Fail-open vs fail-closed—how do you decide?

- **Detailed answer**

- Decide based on **business risk** and **blast radius**:
  - **Fail-closed** for security-critical operations (auth, payments).

- **Fail-open** for non-critical features where availability matters more (recommendations).
  - Always log/metric the fallback mode and alert on it.
- **Example**
  - If Redis for fine-grained rate limiting is down:
    - allow read-only endpoints with coarse WAF limits (fail-open)
    - block login brute-force protections (fail-closed) if policy requires.
- **Common follow-ups**
  - How do you prevent attackers from exploiting fail-open?
  - What is your layered defense strategy?

## 1.2 Amazon design deep-dive follow-ups

- **Idempotency**
    - How do you implement idempotency keys? Where stored? TTL?
    - How do you handle “same key, different payload”?
  - **Retries/Timeouts**
    - How do you avoid retry storms? Where do you place circuit breakers?
  - **Data**
    - Aurora vs DynamoDB: access pattern justification.
    - Schema evolution and zero-downtime changes.
  - **Multi-region**
    - Active-active vs active-passive; RTO/RPO; failover and consistency.
- 

## 2) Amazon — Behavioral (Leadership Principles)

Use **STAR + metrics** and be ready for deep follow-ups.

### 2.1 Question list

- **Ownership**
- **Dive Deep**
- **Bias for Action**
- **Are Right, A Lot**
- **Earn Trust**
- **Disagree and Commit**
- **Invent and Simplify**
- **Deliver Results**

## 2.2 Sample answers (use the master doc templates)

Use the detailed templates in your master file section: - ## 3.4 Detailed sample answers (templates you can tailor)

When practicing, ensure you include: - **Before/after metrics** - **Tradeoffs considered** - **What you learned** - **How you prevented recurrence**

## 2.3 High-signal detailed Q&A (Amazon behavioral)

Use STAR, but always add: - **What was hard/ambiguous?** - **What alternatives did you reject and why?** - **How did you measure success?**

Q1: Ownership — “Tell me about a time you took ownership of a failing system.”

- **Detailed answer (structure)**
  - Situation: define customer impact + metric.
  - Task: what you owned end-to-end.
  - Action: mitigation first, then diagnosis, then prevention.
  - Result: metrics improved + permanent guardrails.
- **Example**
  - “Checkout p99 increased to 3–5s; I led incident response, implemented timeouts/bulkheads, and added SLO-based alarms.”
- **Common follow-ups**
  - What did you do personally?
  - What would you do differently?

Q2: Dive Deep — “Hardest production issue you debugged.”

- **Detailed answer (structure)**
    - Show your hypothesis tree and how you eliminated branches with evidence.
    - Name the artifacts: traces, thread dumps, slow query logs, GC metrics.
  - **Example**
    - “Pool starvation caused request queuing; thread dumps showed blocked threads; fixed by resizing pools and removing an N+1 call.”
  - **Common follow-ups**
    - Why didn’t monitoring catch it earlier?
    - What tests did you add?
- 

## 3) Amazon — Deep Dive Debugging

Amazon often expects **incident command** behavior: - identify blast radius - mitigate fast - gather evidence - prevent recurrence

### 3.1 Question list

- Walk me through your first 15 minutes when p99 latency spikes.
- Connection pool starvation: how do you detect and fix it?
- Memory leak / OOM: how do you confirm and mitigate?
- Kafka/MSK consumer lag: top causes and verification steps.
- Data inconsistency across services: how do you trace the lifecycle?

### 3.2 Answer bullets (how to structure)

- **Signals:** RED metrics + saturation + dependency health
- **Hypotheses:** top 3 likely causes
- **Verification:** traces, logs, thread dumps, heap stats, slow query logs
- **Mitigation:** rate limit, degrade, rollback, disable features
- **Prevention:** alarms, tests, canary gates, runbooks

### 3.3 High-signal detailed Q&A (Amazon debugging)

Q1: “First 15 minutes when p99 latency spikes”

- **Detailed answer**
  - Confirm scope: which endpoints, which AZ/region, which tenants.
  - Check RED + saturation: thread pools, HTTP/DB pools, queue depths.
  - Identify if it correlates with deploy/config.
  - Use traces to find slow spans; verify with dependency metrics.
  - Mitigate: rollback/canary stop, throttle, degrade non-critical calls.
- **Example**
  - p99 spike after deploy: tracing shows downstream payment calls slowed; rollback to previous version + tighten timeouts.
- **Common follow-ups**
  - How do you avoid retry storms?
  - What alarms would have paged you earlier?

Q2: “Kafka consumer lag keeps growing—what do you do?”

- **Detailed answer**
  - Check if lag is due to:
    - insufficient consumers vs partitions
    - slow processing (DB locks, downstream latency)
    - poison messages
    - frequent rebalances
  - Mitigate:
    - scale consumers, add backpressure, isolate poison messages, reduce per-message work.

- **Example**
    - Lag grows only for one partition: hot key. Fix by changing partition key strategy or sharding.
  - **Common follow-ups**
    - How do you ensure idempotency?
    - What is your DLQ/retry policy?
- 

## 4) Google — System Design (what to expect + questions + answer bullets)

Google interviews typically emphasize: - **Correctness** (invariants, consistency, ordering, dedupe) - **Scalability** (partitioning, tail latency) - **Simplicity** (avoid unnecessary components) - **Clear reasoning** and tradeoffs

### 4.1 Common system design prompts

#### Feed / Timeline

- **Questions**
  - Fanout-on-write vs fanout-on-read?
  - How do you handle ranking, freshness, pagination?
- **Answer bullets**
  - Choose strategy based on write/read ratio and latency.
  - Control fanout; use caching; design for p99.

#### *High-signal detailed Q&A (Google)*

#### Q1: Fanout-on-write vs fanout-on-read—how do you choose?

- **Detailed answer**
  - Fanout-on-write:
    - Great for fast reads; expensive writes; complex for celebrities.
  - Fanout-on-read:
    - Cheaper writes; reads become heavy; needs good caching and ranking.
  - Decide using:
    - write/read ratio
    - p99 targets
    - storage budget
    - consistency expectations
- **Example**
  - Typical users: fanout-on-write to precompute timeline.

- Celebrity accounts: hybrid—store posts once; compute follower feeds on read with caching.
- **Common follow-ups**
  - How do you paginate correctly with ranking changes?
  - How do you handle hot keys and load spikes?

## Logging/Auditing Platform

- **Questions**
  - How do you ensure immutability and search?
- **Answer bullets**
  - Append-only store, strict schemas, retention; searchable index.

### *High-signal detailed Q&A (Google)*

#### Q2: How do you ensure immutability and still allow search?

- **Detailed answer**
  - Source of truth: immutable append-only storage.
  - Build derived indices for search.
  - Ensure integrity via checksums/signing (conceptually) and strict access controls.
  - Define retention + GDPR deletion strategy (tombstone vs crypto-shredding).
- **Example**
  - Raw logs in object storage; index in search store. If index is corrupted, reindex from immutable source.
- **Common follow-ups**
  - How do you manage high-cardinality fields?
  - How do you handle multi-tenant isolation?

## Notification / PubSub Systems

- **Questions**
  - Delivery semantics, ordering, replay.
- **Answer bullets**
  - Define delivery guarantees; design idempotent consumers.

## 4.2 Google deep-dive follow-ups

- Define invariants (exactly-once vs effectively-once).
  - Hot keys / hot partitions mitigation.
  - Tail latency strategies across multiple RPC hops.
  - Capacity planning and load testing.
-

## 5) Google — Behavioral

### 5.1 Question list

- Tell me about a time you improved system reliability significantly.
- Describe a technical decision balancing product needs with engineering constraints.
- Tell me about a time you influenced without authority.
- Describe a time you mentored someone to become stronger.
- A time you handled ambiguity and created alignment.

### 5.2 Answer guidance

Use the same stories as Amazon but emphasize: - What was the **technical reasoning**? - What was the **maintainability** story? - What did you do to ensure correctness and simplicity?

### 5.3 High-signal detailed Q&A (Google behavioral)

Q1: “Tell me about a time you improved system reliability significantly.”

- **Detailed answer**
    - Define the SLO/SLI first.
    - Explain the top 2–3 root causes you removed.
    - Explain long-term guardrails (testing, rollout, alerts).
  - **Example**
    - “We set checkout SLO to 99.9% and reduced p99 from 2s to 700ms by removing N+1 calls and adding bulkheads + caching.”
  - **Common follow-ups**
    - How did you prove it was sustained improvement?
    - What did you simplify?
- 

## 6) Google — Deep Dive Debugging

### 6.1 Question list

- Debug a p99 regression across multiple microservices.
- Identify tail latency contributors in a fanout architecture.
- Debug data inconsistency in eventual consistency systems.
- Debug JVM memory/GC behavior and performance bottlenecks.

## 6.2 Answer bullets

- Start from traces; identify slow spans; confirm with metrics.
- Differentiate:
  - GC pauses vs blocking IO vs lock contention
  - dependency slowness vs retry storms
- Propose mitigations + prevention (tests, canary gates, SLO-based alerting)

## 6.3 High-signal detailed Q&A (Google debugging)

Q1: “Debug p99 regression across multiple microservices”

- **Detailed answer**
  - Start with a trace of a slow request and identify the slowest span.
  - Check if it is:
    - downstream dependency
    - queuing/saturation
    - GC pauses
    - lock contention
  - Use correlation IDs to compare baseline vs regression.
  - Fix: reduce fanout, cache, index, tune pools, add backpressure.
- **Example**
  - Traces show the DB span dominates; query plan regression caused by schema change; add index + roll forward.
- **Common follow-ups**
  - How do you design to avoid tail latency amplification?
  - What SLO-based alert would you create?